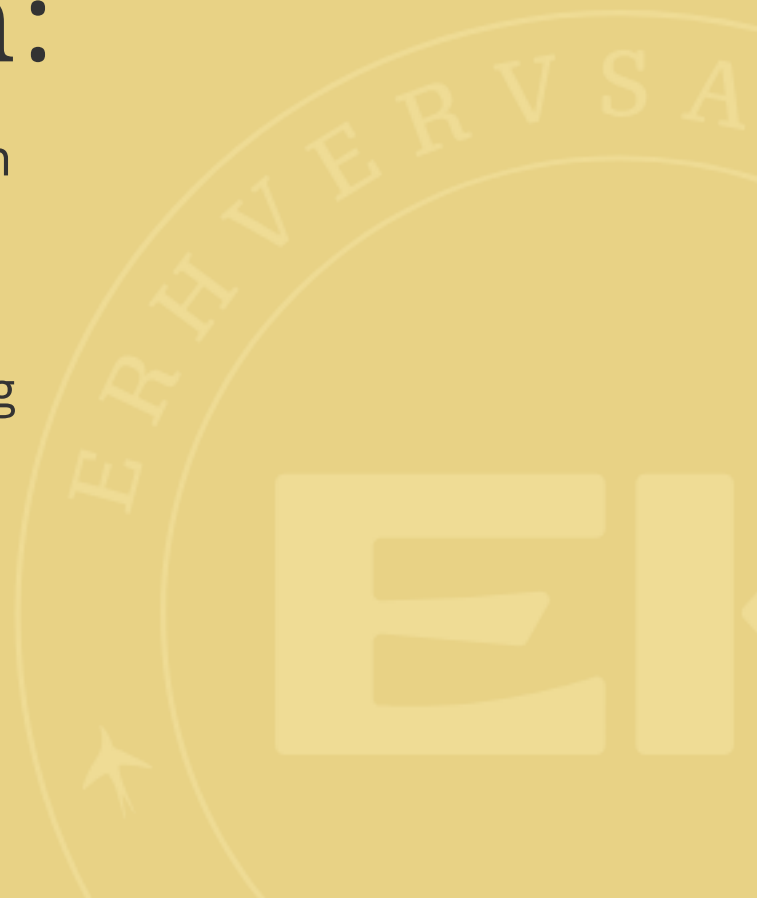


Komposition



Program:

- Lidt repetition
- Komposition
- Øvelser
- Opsummering



A photograph of three people in a meeting. On the left, a man in a white t-shirt looks intently at a laptop. In the center, a woman with blonde hair, wearing a dark blazer over a white top, smiles at the camera. On the right, another person is partially visible, holding a pen. The laptop screen shows a colorful dashboard with various charts and graphs. A white cup and a blue folder are on the table.

Arrays af objekter

Vi kan lave arrays af objekter, fx **BankAccount**-objekter

```
BankAccount[] accounts = new BankAccount[3];  
accounts[0] = new BankAccount(337898);  
accounts[1] = new BankAccount(337899);  
accounts[2] = new BankAccount(337900);  
  
accounts[0].deposit(100);  
System.out.println(accounts[0].balance); // 100.0
```

og vi kan iterere over dem med en for-each løkke

```
for (BankAccount account : accounts) {  
    System.out.println("Saldo på konto " + account.accountNumber + ": " + account.balance);  
}
```


A group of people are gathered around a table in a meeting. A man in a white t-shirt is on the left, looking at a laptop. A woman in a dark blazer is on the right, smiling. Another person is partially visible on the far right. The laptop screen shows a colorful dashboard or map. There are water bottles and a cup on the table.

ကုလ ?

Objekter er reference-typer

En variabel indeholder en reference eller pegepil (eng: pointer) til et objekt i hukommelsen

`null` er en speciel reference-værdi, `null` pointer, der ikke peger (eng: points) på noget objekt

Derfor er det tilladt at initialisere en **BankAccount**-variabel med **null**

```
BankAccount account = null;
```

og det er tilladt at have **null** i et array af objekter

```
BankAccount[] accounts = new BankAccount[3];  
accounts[0] = new BankAccount(337898);  
accounts[1] = null; // ingen konto  
accounts[2] = new BankAccount(337900);
```

Faktisk er alle elementer i et array af objekter **null**, indtil vi sætter dem til noget andet

```
BankAccount[] accounts = new BankAccount[3];  
System.out.println(accounts[0]); // null  
System.out.println(accounts[1]); // null  
System.out.println(accounts[2]); // null
```

Vi skal derfor huske at tjekke for **null**, før vi bruger en variabel, der peger på et objekt, fx

```
BankAccount[] accounts = new BankAccount[3];
accounts[0] = new BankAccount(337898);
accounts[1] = null; // ingen konto
accounts[2] = new BankAccount(337900);

for (BankAccount account : accounts) {
    if (account != null) {
        System.out.println("Saldo " + account.balance);
    }
}
```


NullPointerException (NPE) er en meget almindelig fejl i Java-programmering.

Det sker når vi prøver at bruge en variabel, der er **null**, som om den pegede på et objekt, fx

```
String name = null;  
System.out.println(name.length()); // NullPointerException
```

A.k.a. vi troede name var en **String**, som har en **length()** - men det havde den jo ikke, da den var **null**.

Alligevel er det meget udbredt at bruge **null** til at indikere "ingen værdi", fx i en metode, der skal returnere et objekt

```
public static BankAccount findAccount(int accountNumber) {  
    if ( /* account not found */ ) {  
        return null; // ingen konto fundet  
    }  
    return new BankAccount(accountNumber); // konto fundet  
}
```

A photograph of three people in a meeting. On the left, a man in a white t-shirt looks towards the right. In the center, a woman with blonde hair in a dark blazer looks at the camera with a slight smile. On the right, another person is partially visible, holding a pen over a laptop. The laptop screen shows a colorful dashboard or map. A white text box is overlaid in the center.

Associationer mellem klasser

Kardinalitet

1-1 association (1-1)

Et kursus har én underviser Et **Course**-objekt har én **Teacher**-objekt

```
public class Course {  
    Teacher teacher;  
}
```

1-many association (1-*)

Et akademi har mange studerende Et **Academy**-objekt har mange **Course**-objekter

```
public class Academy {  
    Course[] courses;  
}
```


many-many association (-)

En **Student** kan være tilmeldt mange **Course**-objekter, og et **Course**-objekt kan have mange **Student**-objekter (en studerende kan være tilmeldt mange kurser, og et kursus kan have mange studerende)

```
public class Student {  
    Course[] courses;  
}  
  
public class Course {  
    Student[] students;  
}
```

DEMO Vi giver **BankAccount** en ejer **Owner**

Lad os gøre det obligatorisk at angive en ejer, når vi laver en konto

DEMO: Gør **Owner** obligatorisk for **BankAccount**

Hvad kalder vi det?

Komposition

når et objekt "har et" (**has-a**) andet objekt (stærkt ejerskab)

Komposition - stærkt ejerskab

En konto kan **ikke eksistere uden** en ejer.

Derimod vil en **Student** eksistere fint uden **Course**-objekter

Aggregation

når et objekt "har et" (**has-a**) andet objekt (svagt ejerskab)

... men vi kan nøjes med at kalde det en **has-a** relation

Nævn tre ting du tager med fra i dag?